



Deep Learning, NLP & MLOps – Interview

Q&A



Section 2: Deep Learning & Neural Networks

Q1. What is a neural network and how does it work?

Answer:

A neural network is a series of layered mathematical functions inspired by the brain. Each layer applies:

$$z = W \cdot x + b$$

$$a = f(z)$$

Where:

- W = weight matrix
- b = bias vector
- f = activation function
- a = activated output (passed to next layer)

Training process:

1. **Forward pass** – compute predictions
 2. **Compute loss** – compare prediction vs ground truth
 3. **Backpropagation** – compute gradients via chain rule
 4. **Update weights** – using gradient descent optimizer
-

Q2. What are activation functions? Name common ones.

Answer:

Activation functions introduce **non-linearity** into the network, enabling it to learn complex patterns.

Activation	Formula	Range	Use Case
------------	---------	-------	----------

Activation	Formula	Range	Use Case
Sigmoid	$1/(1+e^{-x})$	(0,1)	Binary output layer
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	(-1,1)	Hidden layers (zero-centered)
ReLU	$\max(0, x)$	$[0, \infty)$	Hidden layers in CNNs, most common
Leaky ReLU	$\max(\alpha x, x), \alpha \approx 0.01$	$(-\infty, \infty)$	Fix dying ReLU problem
Softmax	$e^{x_i} / \sum e^{x_j}$	(0,1), sums to 1	Multi-class output layer
GELU	$x \cdot \Phi(x)$	$(-\infty, \infty)$	Transformers (BERT, GPT)

Dying ReLU problem: Neurons that always output 0 (when $z < 0$) never update. Fixed with Leaky ReLU or ELU.

Q3. What is backpropagation?

Answer:

Backpropagation is the algorithm used to compute gradients of the loss function with respect to all weights in the network, using the **chain rule of calculus**.

Chain rule:

$$\partial L / \partial w_1 = (\partial L / \partial a_2) \cdot (\partial a_2 / \partial z_2) \cdot (\partial z_2 / \partial a_1) \cdot (\partial a_1 / \partial z_1) \cdot (\partial z_1 / \partial w_1)$$

Steps:

1. Forward pass – compute all activations
2. Compute loss at the output
3. Backward pass – propagate gradients from output to input layer
4. Update weights: $w = w - \alpha \cdot \partial L / \partial w$

Q4. What is vanishing/exploding gradient problem?

Answer:

Vanishing gradients: Gradients become extremely small as they propagate backward through many layers. Early layers learn very slowly. Common with sigmoid/tanh in deep networks.

Exploding gradients: Gradients grow exponentially large. Causes weight updates to be unstable (NaN values).

Solutions:

Problem	Solutions
Vanishing	ReLU, Batch Normalization, ResNet skip connections, LSTM/GRU gates
Exploding	Gradient clipping, weight initialization (Xavier/He), lower learning rate

Q5. What is dropout and how does it work?

Answer:

Dropout is a regularization technique where, during training, each neuron is **randomly set to zero** with probability p (typically 0.2–0.5).

Effect:

- Prevents neurons from co-adapting
- Acts like training an ensemble of many different networks
- At inference time, all neurons are active but outputs are scaled by $(1-p)$

```
nn.Dropout(p=0.5) # PyTorch
```

Q6. What is Batch Normalization?

Answer:

Batch Normalization normalizes the input of each layer so that it has **zero mean and unit variance** across the mini-batch.

$$\hat{x}_i = (x_i - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)}$$
$$y_i = \gamma \cdot \hat{x}_i + \beta \quad (\gamma, \beta \text{ are learnable})$$

Benefits:

- Reduces internal covariate shift
 - Allows higher learning rates
 - Acts as a mild regularizer
 - Reduces sensitivity to weight initialization
-

Q7. What is CNN (Convolutional Neural Network)?

Answer:

CNNs are designed for **grid-like data** (images, sequences). Key components:

Layer	Operation	Purpose
Convolutional	Applies learnable filters	Detect local patterns (edges, textures)
Activation (ReLU)	Element-wise non-linearity	Introduce non-linearity
Pooling (Max/Avg)	Down-samples feature maps	Reduce spatial dimensions, translation invariance
Fully Connected	Standard neural network layer	Classification/regression output

Key terms:

- **Kernel/Filter size** – e.g., 3×3, 5×5
- **Stride** – step size of the filter
- **Padding** – add zeros around border to control output size

Output size formula:

$$\text{Output} = (W - F + 2P) / S + 1$$

Where W=input size, F=filter size, P=padding, S=stride

Q8. What is RNN and why are LSTMs/GRUs better?

Answer:

RNN (Recurrent Neural Network): Processes sequential data by maintaining a hidden state:

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

Problem: Suffers from **vanishing gradients** over long sequences – can't capture long-term dependencies.

LSTM (Long Short-Term Memory): Solves this with gates:

- **Forget gate** – what to discard from cell state
- **Input gate** – what new info to store
- **Output gate** – what to output
- **Cell state** – long-term memory highway

GRU (Gated Recurrent Unit): Simplified LSTM with only 2 gates (update + reset). Faster to train, similar performance.

Q9. What is a Transformer? How does self-attention work?

Answer:

Transformer is an architecture based entirely on **attention mechanisms** (no RNNs). Foundation for BERT, GPT, T5, etc.

Self-Attention mechanism:

For each token, compute:

$$Q = XW_Q \quad (\text{Query})$$

$$K = XW_K \quad (\text{Key})$$

$$V = XW_V \quad (\text{Value})$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

- $Q \cdot K^T$ – similarity between tokens (attention score)
- $\sqrt{d_k}$ – scaling factor to prevent gradient vanishing
- **softmax** – converts scores to probabilities
- Weighted sum of V – weighted by attention scores

Multi-head attention: Runs attention h times in parallel with different learned projections, then concatenates.

Why Transformers beat RNNs:

- Parallel computation (no sequential bottleneck)
 - Captures long-range dependencies directly
 - Scales better with data and compute
-

Q10. What is transfer learning?

Answer:

Transfer learning uses a model **pre-trained on one task/dataset** and fine-tunes it for a different (but related) task.

Why it works: Low-level features (edges, textures, word co-occurrences) are generalizable.

Strategies:

1. **Feature extraction** – freeze pre-trained layers, only train final classifier
2. **Fine-tuning** – unfreeze some/all layers and train with small learning rate

Examples:

- Computer Vision: ResNet, VGG, EfficientNet (pre-trained on ImageNet)
- NLP: BERT, RoBERTa, BanglaBERT (pre-trained on large text corpora)

Your research connection: You used transformers for Bangla NLP and real-time vision systems – always connect your research here!

 Section 3: NLP Questions

Q11. What is the difference between BoW, TF-IDF, and embeddings?

Answer:

Method	Description	Captures Semantics?	Sparse/Dense
BoW	Count of each word in vocab	No	Sparse
TF-IDF	Weighs words by frequency & rarity	No	Sparse
Word2Vec	Word → dense vector via co-occurrence	Yes	Dense
FastText	Like Word2Vec + subword characters	Yes (OOV handled)	Dense
BERT	Contextual embeddings from transformer	Yes (context-aware)	Dense

TF-IDF formula:

$$TF\text{-}IDF(t, d) = TF(t, d) \times \log(N / df(t))$$

Where TF = term frequency in doc, N = total docs, df = docs containing term.

Your research connection: You used BoW, TF-IDF, character n-grams, Word2Vec, and FastText for Bangla – excellent practical experience!

Q12. What is agglutinative morphology and why does it matter for Bangla NLP?

Answer:

Agglutinative languages form words by adding **multiple suffixes/prefixes** to a root. This means a single root word can generate thousands of surface forms.

Challenge for NLP:

- Standard tokenizers produce huge vocabularies
- Sparse representations (BoW/TF-IDF) suffer more
- Word2Vec sees rare word forms rarely enough to learn poor embeddings

Solutions:

- **Subword tokenization** (BPE, WordPiece used by BERT) – handles unseen word forms
- **FastText** – trains on subword character n-grams, handles OOV naturally
- **Morphological analyzers** – explicit stemming/lemmatization
- **Character-level features** – capture morphological patterns

Your research connection: This is directly from your publication! Explain how you used character n-grams and sparse feature combinations to overcome this.

Q13. What is the difference between stemming and lemmatization?

Answer:

	Stemming	Lemmatization
Approach	Rule-based suffix stripping	Dictionary-based, finds root form
Output	May not be a real word ("running" → "run", "studies" → "studi")	Always a valid word ("running" → "run", "studies" → "study")
Speed	Fast	Slower
Use case	Information retrieval	NLP tasks requiring linguistic accuracy

Q14. What is Named Entity Recognition (NER)?

Answer:

NER identifies and classifies **named entities** in text into predefined categories:

- **PER** – Person names
- **ORG** – Organizations
- **LOC** – Locations
- **DATE, MONEY**, etc.

Approaches:

- Rule-based

- CRF (Conditional Random Fields)
- BiLSTM-CRF
- Transformer-based (BERT fine-tuned on NER corpora)

Section 4: MLOps Questions

Q15. What is MLOps and why is it important?

Answer:

MLOps (Machine Learning Operations) applies DevOps principles to ML systems to ensure **reliable, scalable, and reproducible** model development and deployment.

Key pillars:

- **Versioning** – track data, code, and model versions (Git, DVC, MLflow)
- **CI/CD for ML** – automated testing and deployment pipelines
- **Model monitoring** – detect performance degradation in production
- **Feature stores** – centralized, reusable feature engineering
- **Model registry** – manage and deploy model versions

Q16. What is data drift and concept drift?

Answer:

Type	Definition	Example
Data drift (covariate shift)	Input feature distribution $P(X)$ changes	Sensor readings change after hardware upgrade
Concept drift	Relationship $P(Y X)$ changes	Customer preferences change over time
Label drift	Target distribution $P(Y)$ changes	More fraud cases appear in new season
Model decay	Performance degrades over time	Combined effect of above

Detection methods:

- Statistical tests (KS-test, Population Stability Index)
- Monitor prediction distributions
- Track key metrics over time windows

Solutions: Retrain periodically, use online learning, set up alerting systems.

Q17. What is a machine learning pipeline?

Answer:

An ML pipeline is an automated sequence of steps from raw data to deployed model:

Data Ingestion → Data Validation → Preprocessing → Feature Engineering
→ Model Training → Model Evaluation → Model Registry → Deployment → Monitoring

Tools:

- **scikit-learn Pipeline** – chain preprocessing + model
 - **Apache Airflow / Prefect** – workflow orchestration
 - **Kubeflow** – ML pipelines on Kubernetes
 - **Azure ML Pipelines** – cloud ML orchestration
 - **MLflow** – experiment tracking + model registry
-

Q18. What is Azure Machine Learning?

Answer:

Azure Machine Learning (Azure ML) is Microsoft's cloud platform for the ML lifecycle:

Feature	Description
Compute clusters	Scale training on GPU/CPU VMs
Datasets	Versioned, registered datasets
Experiments	Track runs, metrics, logs
Model Registry	Version and manage trained models
Endpoints	Deploy models as REST APIs
Pipelines	Orchestrate multi-step ML workflows
AutoML	Automated model selection and tuning

Key SDK:

```
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential
```

```
ml_client = MLClient(DefaultAzureCredential(), subscription_id, resource_
```

Q19. How do you deploy a machine learning model?

Answer:

Deployment options:

Method	Description	Use Case
REST API (Flask/FastAPI)	Wrap model in web API	General purpose
Docker container	Package model + dependencies	Portable, reproducible
Kubernetes	Orchestrate containers at scale	High availability
Cloud endpoints	Azure ML, AWS SageMaker, GCP Vertex	Managed, scalable
Edge deployment	ONNX, TFLite on device	Real-time, offline

Basic Flask deployment:

```
from flask import Flask, request, jsonify
import pickle

app = Flask(__name__)
model = pickle.load(open('model.pkl', 'rb'))

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['features']
    prediction = model.predict([data])
    return jsonify({'prediction': prediction[0]})
```

Q20. What is model monitoring? What metrics do you track?

Answer:

Model monitoring tracks model performance **after deployment** to detect degradation.

Metrics to monitor:

- **Prediction distribution** – detect shifts in output

- **Feature distributions** – detect data drift
- **Business KPIs** – revenue impact, conversion rates
- **Technical metrics** – latency, throughput, error rates
- **Ground truth metrics** – accuracy, F1 (when labels come in)

Tools: MLflow, Evidently AI, Seldon, WhyLogs, Azure Monitor

Q21. What is the difference between online and batch inference?

Answer:

	Batch Inference	Online (Real-time) Inference
When	Scheduled (hourly, daily)	Immediately on request
Latency	High (minutes to hours)	Low (milliseconds)
Throughput	High	Lower per request
Use case	Recommendation pre-computation, report generation	Fraud detection, chatbots, search ranking
Infrastructure	Spark, batch jobs	REST API, streaming

Q22. What is Docker and why is it used in MLOps?

Answer:

Docker is a containerization platform that packages applications and their dependencies into **portable, isolated containers**.

Why for ML:

- **Reproducibility** – same environment everywhere (dev, staging, prod)
- **Dependency management** – no "works on my machine" issues
- **Scalability** – containers can be replicated easily
- **CI/CD integration** – automated build and push pipelines

Basic Dockerfile for ML:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
```

COPY . .

CMD ["python", "app.py"]
